

WebSpector - An Agentic AI for Automated Website QA

Final Year

FYP Proposal

Session 2022-2026

A project submitted in partial fulfilment of the
requirements for the degree of
Bachelor of Science in
Computer Science



Department of Computer Science

Namal University Mianwali

15 October 2025

Project Registration

Project ID (for office use)						
Type of project	☐ Traditional		☐ Industrial		☐ Continuing	
Nature of project	☐ Development		☐ Research		☐ R&D	
Area of specialisation						
Project Group Members						
Sr.#	Reg. #	Student Name	CGPA	Email ID	Phone #	Signature
(i)	NUM-BSCS-2022-42	Jalal Ur Rahman	2.96	bscs22f42@namal.edu.pk	03425481024	
(ii)	NUM-BSCS-2022-48	Sultan Haider	3.08	bscs22f48@namal.edu.pk	03311204792	
Declaration: FYP group members have cleared all prerequisites courses (mentioned below) for FYP-I as per their degree requirements. (All core courses)						

Plagiarism Free Certificate

This is to certify that, I am _____ S/D/o _____, group member of FYP under registration no _____ at Computer Science Department, Namal University. I declare that my FYP proposal is checked by my supervisor and the similarity index is _____% that is less than 20%, an acceptable limit by HEC. Report is attached herewith as Appendix A.

Date: _____

Name of Group Member 1: _____ Signature: _____

Name of Group Member 2: _____ Signature: _____

Name of Supervisor: _____

Signature: _____

Project Abstract

Modern web applications are increasingly dynamic, adaptive, and complex, which makes testing and quality assurance (QA) highly challenging. Traditional QA tools that rely on predefined scripts or manual test cases often fail to detect new visual or functional issues that arise from frequent updates and responsive layouts, while manual QA is slow, error prone, and resource intensive. WebSpector is an intelligent, AI powered website testing framework that automates the QA process without human written scripts by combining Large Vision Models (LVMs) and Large Language Models (LLMs) to interact with websites, analyze layouts, detect interface anomalies, and generate detailed natural language bug reports. The system will use browser automation tools such as Selenium or Playwright to navigate sites and capture screenshots across multiple devices and resolutions; these screenshots will be processed by models like BLIP-2, CLIP, or GPT-4V for visual understanding and anomaly detection, and an AI reasoning engine (e.g. GPT-4 or LLaMA) will summarize findings into structured QA reports. The final deliverable will be a web based tool where users submit any website URL and receive a comprehensive, AI generated testing report making web QA faster, more reliable, and more cost effective.

1 Introduction

In the modern era of web development, **Quality Assurance (QA)** plays a crucial role in ensuring that websites deliver consistent, reliable, and visually coherent user experiences. As websites have evolved into dynamic, data-driven, and responsive systems, the process of testing and maintaining their quality has become increasingly complex. Frequent design updates, adaptive layouts, and user specific interfaces make it difficult to verify whether all features and visual components continue to function as expected across devices and browsers.

Traditional QA approaches rely heavily on **manual testers** or **rule based automation frameworks** such as *Selenium* and *Cypress*. Although these tools enable automation of predefined test cases, they suffer from limited flexibility whenever a webpage's structure or layout changes, the test scripts must be manually updated. Manual testing, on the other hand, is resource intensive, time-consuming, and prone to human error. Consequently, these conventional techniques often fail to detect new or context dependent issues, especially in large scale or frequently updated websites.

Recent progress in **Artificial Intelligence (AI)** offers a transformative opportunity to reimagine the QA process. Modern **Large Vision Models (LVMs)** and **Large Language Models (LLMs)** such as BLIP-2, CLIP, GPT-4V, and LLaMA possess remarkable abilities to interpret visual information, understand layouts, and generate meaningful descriptions of visual scenes. When combined with **agentic AI frameworks** like LangChain or Auto GPT, these models can autonomously reason, plan, and execute multi step tasks. This shift from rule based automation to **cognitive automation** allows testing systems to act more like human testers observing, analyzing, and reasoning about what they see.

The proposed project, **WebSpector - An Agentic AI for Automated Website QA**, aims to address the limitations of traditional testing by integrating **vision language understanding** and **autonomous reasoning** into a single intelligent framework. Instead of depending on

fixed test scripts, WebSpector will employ AI agents that can interact with web interfaces, visually inspect pages, identify potential anomalies, and report them in natural language. This approach aligns with the current evolution of intelligent systems capable of **self-directed problem solving and multimodal reasoning**.

By introducing AI-driven perception and reasoning into website QA, WebSpector seeks to make testing **faster, adaptive, and more accurate**, significantly reducing human effort while improving defect detection in complex web environments. The system’s development will also contribute to exploring how AI models can replace traditional scripted methods with observation based, context aware validation paving the way for the next generation of **autonomous software quality assurance systems**.

2 Related work

Browser automation frameworks such as Selenium and Playwright form the foundation of contemporary end-to-end (E2E) website testing. Selenium implements the W3C WebDriver standard and provides language bindings across platforms, enabling cross browser automated actions. Playwright offers a modern API designed for reliable multi context testing and provides native support for Chromium, WebKit, and Firefox. Both tools are widely used in industry for regression testing and continuous integration pipelines however, they remain script driven and require frequent maintenance when web interfaces change [1].

Visual regression testing compares rendered screenshots against baselines to detect UI level changes. Commercial platforms such as Applitools Eyes and Testim use machine learning based heuristics, self healing selectors, and cloud based cross browser grids to reduce false positives and improve testing stability. Although these systems improve upon traditional pixel difference techniques, they still rely on human defined baselines or scripted workflows to test application states, limiting their adaptability and autonomy [2].

In academic research, various studies have explored visual GUI testing and automated detection of visual bugs. Early work demonstrated that screenshot comparison can serve as a practical oracle for verifying GUI correctness [3]. Subsequent research introduced model based and delta detection techniques to identify visual interface changes, as well as approaches for specialized graphics such as canvas based or procedural rendering [4]. Other studies proposed frameworks to enhance detection accuracy in GUI based web systems [5]. Recent investigations further expanded this area by applying visual inference models to automate GUI testing tasks, showing strong results in identifying subtle design regressions [6]. While these works demonstrate the feasibility of automated visual bug detection, many rely on assumptions such as labeled datasets or rigid application structures, which limit their generalization to dynamic, responsive websites.

Recent advancements in multimodal AI, particularly in vision language models, have enabled deeper semantic understanding of images and web layouts. BLIP-2 introduced an efficient framework for language image pretraining that uses frozen vision encoders combined with large language models to achieve strong zero shot image-to-text reasoning [7]. Similarly, multimodal systems like GPT-4V exhibit high level perception and reasoning capabilities, allowing QA tools to move beyond simple pixel level comparisons toward

semantic understanding of screenshots such as identifying missing product images or misplaced buttons.

Agentic AI frameworks, including LangChain and Auto GPT, have made it possible for large language models to orchestrate external tools, perform multi step reasoning, and plan automated actions. These frameworks enable an LLM to control browser automation tools, analyze visual data, and decide subsequent actions, creating the foundation for autonomous, closed loop website QA systems [8]. However, there is limited published work integrating multimodal perception with agentic reasoning to build fully automated web testing agents.

In summary, current industrial solutions excel at scripted regression or baseline based testing, while academic research demonstrates strong performance under controlled settings. The recent convergence of multimodal AI and agentic frameworks presents a unique opportunity to achieve autonomous, semantic QA workflows. WebSpector addresses this gap by combining a browser interaction engine, a vision language perception layer, and an agentic planner into a unified pipeline for script free, observation driven QA. This approach aims to make web testing more adaptive, scalable, and human like in reasoning [2]. Furthermore, it leverages semantic understanding and cognitive reasoning capabilities introduced by modern multimodal AI systems to improve the quality and efficiency of automated web testing [7].

3 Project Rationale

Quality Assurance (QA) is a critical part of modern web development, ensuring that websites behave correctly, display consistently, and provide seamless usability across devices. However, traditional QA processes are becoming inefficient for the scale and speed of today's web systems. Studies indicate that **nearly 35 to 40% of total software development time** is consumed by repetitive testing cycles, much of which involves manually verifying layouts, forms, and navigation flows.

The repetitive nature of manual testing and the fragility of rule-based automation scripts significantly slow down release cycles and increase project costs. Each modification in a website's layout or structure often requires test scripts to be updated, which defeats the purpose of automation. Furthermore, current tools although robust still depend on **human defined test cases**, making them incapable of detecting previously unseen or unexpected interface issues.

This project proposes **WebSpector - An Agentic AI for Automated Website QA**, an innovative solution that leverages **Large Vision Models (LVMs)**, **Large Language Models (LLMs)**, and **Agentic AI frameworks** to perform website testing autonomously. By combining visual perception and language reasoning, the system will be able to inspect, understand, and evaluate web pages without human-authored scripts. This reduces human effort, enhances test coverage, and enables the system to adapt to new web layouts and designs automatically.

Ultimately, the project aims to revolutionize traditional QA workflows by transforming them from static, rule-based methods into **dynamic, observation driven testing systems** powered by cognitive AI. This transition will save time, improve testing accuracy, and

ensure higher software reliability particularly for large, responsive, and continuously evolving web applications.

his project focuses on improving the way quality assurance works right now. Most of the time spent testing software up to 40% of the entire development process is used for doing the same checks over and over. Using an AI that can act on its own can cut down this time a lot by automatically checking and testing websites.

Our goal is to build an AI that works like a QA engineer, spotting visual and functional issues without needing specific test steps. This method helps developers and testers save time, cut costs, and keep the quality the same on all devices.

3.1 Aims and Objectives

Aim: To develop an **AI-powered autonomous website QA framework** that intelligently interacts with, analyzes, and evaluates websites both visually and functionally without requiring manual test scripts.

Objectives:

1. **Automate Browser Interaction:** Implement autonomous website navigation using *Selenium* or *Playwright* to simulate user behavior (clicks, form inputs, page transitions).
2. **Visual Analysis via LVMs:** Use vision-language models such as *BLIP-2*, *CLIP*, or *GPT-4V* to identify UI inconsistencies, misplaced components, and visual layout issues.
3. **AI-based Reasoning and Reporting:** Integrate *GPT-4* or *LLaMA* via *LangChain* agents to interpret detected anomalies and generate detailed, natural-language QA reports.
4. **Dashboard Visualization:** Develop an interactive web dashboard (Flask based) to display screenshots, test summaries, and defect severity ratings.
5. **Performance Benchmarking:** Evaluate the system's performance in terms of detection accuracy, speed, and reliability compared to conventional QA automation frameworks.

3.2 Scope of the Project

The proposed system will focus exclusively on **automated website quality assurance**, emphasizing intelligent visual and functional testing.

In-scope activities include:

- Evaluating both the **visual appearance** and **interactive behavior** of web applications.
- Ensuring **cross device compatibility** by testing on different viewport sizes (desktop, tablet, mobile).
- Generating **AI written QA reports** with issue descriptions, screenshots, and severity classifications.

- Comparing the efficiency and accuracy of the system with existing manual and automated tools.

Out-of-scope elements:

- Native **mobile or desktop application testing** (not web based).
- Server side code validation or backend performance benchmarking.
- Deep security or penetration testing.

The project will span **two academic semesters**, focusing first on system design and prototype development, followed by model integration, evaluation, and performance validation.

3.3 Success Criterion

The success of WebSpector will be determined based on measurable and verifiable outcomes:

Criterion	Success Indicator
Autonomy	System performs complete QA tests without human-authored scripts.
Detection Accuracy	Detects $\geq 80\%$ of UI and functional defects identified by human testers.
Performance Efficiency	Reduces overall testing time by at least 50% compared to manual testing.
Report Quality	AI-generated reports are comprehensible, well-structured, and correctly identify issue severity.
Cross-Device Consistency	System maintains consistent accuracy across different screen sizes and devices.

Successful achievement of these metrics will validate the project objectives and demonstrate the practicality of agentic AI for QA automation.

3.4 Proposed Methodology and Architecture

The proposed system will employ a **modular and layered architecture**, integrating multiple AI components to simulate the behavior of a human QA engineer. The overall workflow is divided into the following key layers:

1. Web Interaction Layer

- Implements automated website traversal using **Playwright** or **Selenium WebDriver**.

- Simulates real user actions: navigating links, clicking buttons, filling forms, scrolling, and capturing DOM snapshots.
- Takes **screenshots at various viewport resolutions** (e.g., 360×640, 768×1024, 1366×768) to test responsive design consistency.
- Manages browser sessions and collects metadata such as load time, network requests, and page structure.

2. Perception Layer (Vision Analysis)

- Processes the captured screenshots using **Large Vision Models (LVMs)** such as *BLIP-2*, *CLIP*, or *GPT-4V*.
- Detects **visual anomalies**, **missing elements**, and **misalignments** by comparing semantic understanding of the page against layout expectations.
- Extracts high-level features like color contrast, text visibility, and alignment consistency to ensure **UI/UX compliance**.
- Provides semantic descriptions of each screenshot for reasoning in the next layer.

3. Cognitive Reasoning Layer

- Utilizes **Large Language Models (LLMs)** such as *GPT-4* or *LLaMA* integrated via *LangChain* to interpret perception outputs.
- Performs multi-step reasoning to:
 - Identify whether detected visual differences represent genuine defects or acceptable variations.
 - Determine issue categories (UI, navigation, responsiveness, or functional).
 - Recommend actions such as “re-test,” “skip,” or “flag defect.”
- Enables **agentic behavior** where the AI autonomously plans the next testing step (e.g., explore another page or retake a screenshot).

4. Reporting and Storage Layer

- Generates structured, human readable QA reports including:
 - **Issue summary**, **severity level**, **affected components**, and **evidence (screenshots + DOM snippet)**.
- Reports stored in **PostgreSQL** or **SQLite** database for further analytics.
- Supports report export in PDF/CSV format and dashboard visualization.

5. Dashboard and Visualization Layer

- Implements a **Flask based web dashboard** for real time monitoring of test sessions.
- Displays website screenshots, test progress, and AI-generated defect annotations.

- Provides filtering and search options for defect category, severity, and test timestamp.
- Supports comparative analysis between traditional QA methods and WebSpector's output.

6. Evaluation Layer

- Compares WebSpector's results with manually labeled ground truth data.
- Calculates **precision**, **recall**, **F1 score**, and **false positive rate** for quantitative evaluation.
- Measures **test authoring time**, **execution time**, and **bug-detection efficiency** against existing automation frameworks.
- Produces summary charts of performance improvements for inclusion in the final report.

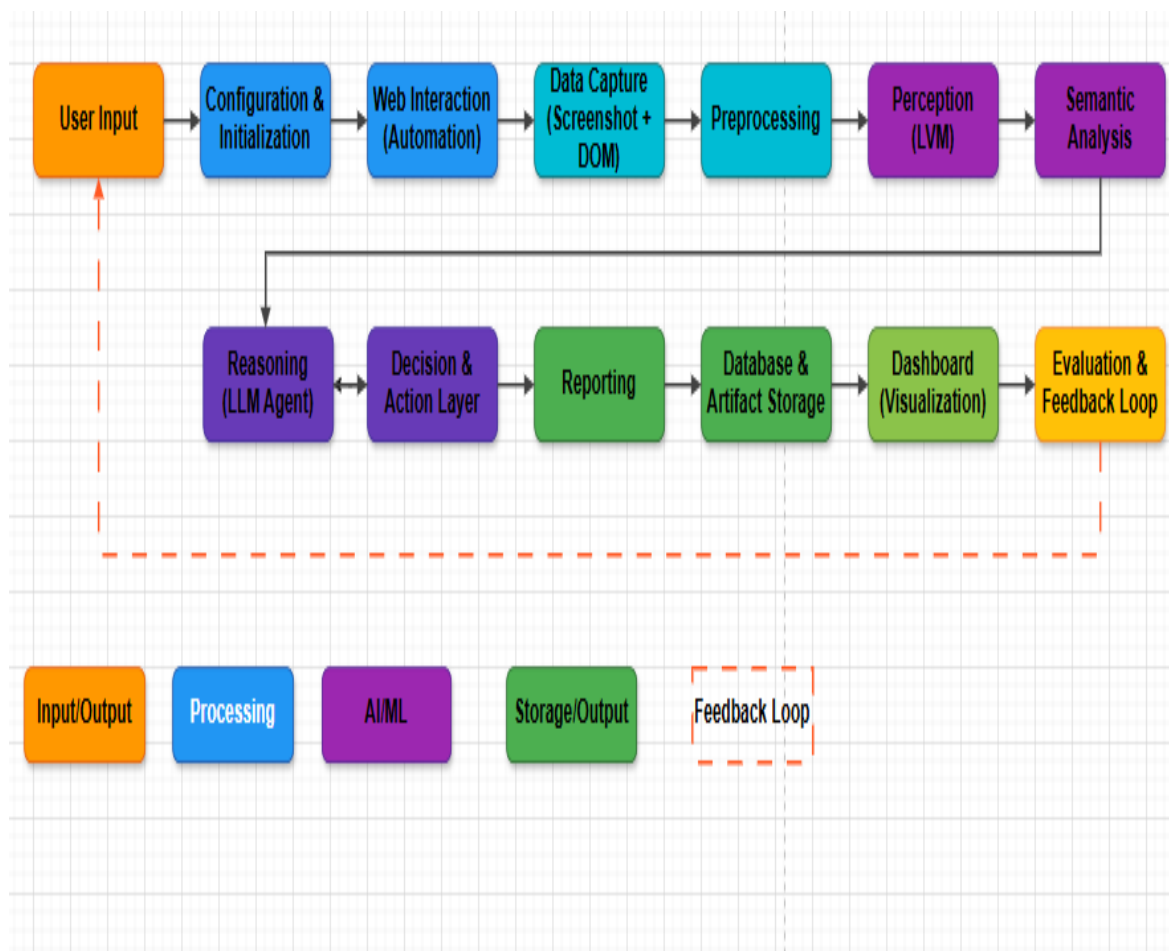


Figure 1: Archeticture Flow

4 Individual Tasks

The expected individual task list with the tentative plan is given below.

Table 1: Individual Task

Team Member	Activity / Responsibility	Oct 2025	Nov 2025	Dec 2025	Jan 2026	Feb 2026	Mar 2026	Apr 2026
Jalal Ur Rahman	Research on agentic AI, vision-language models, and web-testing methodologies	X	X					
	Selenium/Playwright automation setup, browser-driver configuration, and screenshot-capture implementation		X	X				
	Large Vision Model integration (CLIP/BLIP-2/GPT-4V) and visual-analysis pipeline development			X	X			
	API design and backend integration connecting all system components				X	X		
	System testing, performance optimization, and bug fixing					X	X	
	Final integration, documentation, and deployment preparation						X	X
Sultan Haider	Research on UI/UX testing best practices and accessibility standards	X	X					
	UI-element detection algorithms and layout-analysis implementation		X	X				
	Large Language Model integration for reasoning and report generation			X	X			

	Web dashboard frontend development using Flask (HTML/CSS/JavaScript)				X	X		
	Report formatting, visualization components, and export functionality					X	X	
	Comprehensive testing, user documentation, and final presentation materials						X	X
Both Members	Joint evaluation, performance analysis, and user testing				X	X	X	X
	Report compilation and documentation (throughout the project)	X	X	X	X	X	X	X

5 Tools and Technologies

The development of WebSpector- An Agentic AI for Automated Website QA requires a combination of automation frameworks, AI models, libraries, and visualization tools. The following tools and technologies will be employed throughout the project:

- **Programming Language:** Python chosen for its extensive AI ecosystem, support for automation libraries, and integration with modern deep learning frameworks.
- **Frameworks:** Selenium and Playwright for browser automation, user interaction simulation, and multi device screenshot capture. Flask for building the web-based dashboard and API backend.
- **AI Models:** GPT-4V, BLIP-2, and CLIP for vision-language understanding and visual anomaly detection. GPT-4 or LLaMA for natural-language reasoning, report generation, and agentic task orchestration.
- **Libraries:** OpenCV for image preprocessing and layout analysis. LangChain for agentic reasoning and tool coordination. HuggingFace Transformers for model integration and inference.
- **Database:** SQLite (for local testing) or PostgreSQL (for scalable deployment) to store test results, bug reports, screenshots, and metadata.
- **Visualization and Reporting:** Matplotlib and Flask dashboard components using HTML, CSS, and JavaScript for presenting analytics, bug summaries, and evaluation metrics interactively.

6 Gantt Chart

Team	Activity	Duration	Oct	Nov	Dec	Jan	Feb	Mar	Apr
Jalal Ur Rahman	Agentic AI and web-testing research	2							
Sultan Haider	UI/UX testing and accessibility research	2							
Jalal Ur Rahman	Automation setup using Selenium/Playwright	2							
Sultan Haider	UI element detection and layout analysis	2							
Jalal Ur Rahman	Vision model integration and visual pipeline setup	2							
Sultan Haider	Language model integration for reasoning/reporting	2							
Jalal Ur Rahman	Backend API design and system integration	2							
Sultan Haider	Frontend dashboard development (Flask/HTML/CSS/JS)	2							
Jalal Ur Rahman	System testing and performance optimization	2							
Sultan Haider	Reporting, visualization, and user documentation	2							
Jalal Ur Rahman	Final integration, documentation, and deployment	2							
Both	Joint evaluation, performance analysis, and user testing	4							
Both	Continuous report compilation and documentation	7							
Legend:									
Jalal									
Sultan									
Both									

Figure 2: Gantt Chart

7 Bibliography

- [1] Selenium Project, “WebDriver Selenium Documentation,” 2024. [Online]. Available: <https://www.selenium.dev/documentation/>.
- [2] AppliTools, “AppliTools Eyes Visual AI Testing,” 2024. [Online]. Available: <https://applitools.com/products/eyes/>.
- [3] A. Memon, A. Banerjee and A. Nagarajan, “GUI Testing: Systematic Exploration of Event Driven Software,” *IEEE Transactions on Software Engineering*, vol. 31, pp. 884-896, 2005.
- [4] F. Leotta, D. Clerissi, A. Marchetto and F. Ricca, “Visual Regression Testing of Web Applications,” in *Proceedings of the IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, 2013.
- [5] S. Mahajan, A. Sangwan and R. Bhatia, “An Automated GUI Testing Framework for Web Applications,” *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 9, no. 2, pp. 123-129, 2018.
- [6] H. Zhang, Y. Zhou and J. Zhao, “Automated GUI Testing for Web Applications via Visual Inference,” *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 30, no. 1, pp. 1-32, 2021.
- [7] D. L. S. S. H. J. Li, “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models,” *arXiv*, vol. abs/2301.12597, 2023.
- [8] LangChain Team, “Agents LangChain Documentation,” 2024. [Online]. Available: <https://python.langchain.com/docs/modules/agents/>.

10% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **17 Not Cited or Quoted 9%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **1 Missing Citation 1%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 10%  Internet sources
- 3%  Publications
- 0%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.